Accounting Firm Portal — Complete Build Plan & Product Context

MCRC Tax & Accounting · Philippine accounting firm **Live:** https://acctgfirm.mcrctas.com **Document date:** September 2026 **Purpose:** a single self-contained briefing on what this software *is*, what has been **built**, what is **planned**, and how it is **operated**. Written to be handed to another Claude session (or another engineer) as complete context with no prior knowledge.

Table of contents

1. What this software is

A **multi-tenant web application an accounting firm runs for its own practice.** It is not a SaaS product being sold; MCRC Tax & Accounting built it to run MCRC Tax & Accounting.

It does five things:

1. **Client management** — onboard and maintain the firm's client roster, including their BIR registration facts (TIN, RDO, line of business, tax regime, tax types).
2. **Bookkeeping** — capture each client's **sales/income** and **purchases/expenses**, classified for Philippine tax purposes (VAT class, input-VAT category, attribution).
3. **Tax estimation** — turn that classified bookkeeping into a *management estimate* of what the client will owe.
4. **BIR form generation** — compute the **authoritative** figures for nine BIR forms and export **eBIRForms-compatible XML** (or a print-ready PDF for the two certificates).

5. **Client self-service** — a read-scoped **Client Portal** where the client's own people see their sales, expenses, tax position, and filings.

Alongside that it runs the firm's **service catalogue and billing**, a **Chart of Accounts**, a **Financial Statement Creator**, and a machine-access **MCP** endpoint.

Explicitly out of scope

- **Direct e-filing to the BIR.** The Portal produces the XML; a human uploads it through the official eBIRForms package.
- **Payment processing.** Billings are issued and tracked; money moves outside the system. (A Maya gateway was investigated and sandbox-tested — no code shipped.)
- **A full double-entry general ledger.** The Chart of Accounts and FS Creator work from transaction data and trial-balance entries, not from journal postings.

---

2. Who uses it (actors & roles)

Two **user types**, each with its own set of roles and its own navigation shell.

Firm users (`userType = FIRM`)

Role	What they can do
Super Admin	Everything, including `Clients:ViewAll` (see every client without assignment), role configuration, integration clients, audit logs, email settings.
Manager	Clients, categories, sales, expenses, tax computation, billing, BIR forms, invitations, reports, filings, input-tax assets, audit logs — **only for assigned clients**.
Accountant	Bookkeeping + tax + BIR forms + billing + Chart of Accounts + Financial Statements + tax rules; read-only on clients — **assigned clients only**.
Staff	Create/read/import/export sales and expenses; read clients and categories.
Auditor	Read-only across clients, transactions, tax computation, reports, filings, and the audit log.

Client users (`userType = CLIENT`)

Scoped to a single client organisation.

Role	What they can do
Client Owner	Full client-scope: manage their own portal users, create/edit/export their sales and expenses, read tax computation, reports, filings.
Client Manager	Read + export their org's sales, expenses, tax, reports, filings.
Client Staff	(Narrower capture/read set within their org.)

How authorisation actually works

RBAC is **data-driven**, not hard-coded. `permissions` and `roles` are database tables; `permissions.constants.ts` only *seeds and documents* the catalog. Every request is checked twice:

1. **Permission check** — does the caller hold `Resource:Action`?

2. **Per-client scope check** — is the caller *assigned* to the client the request touches? `FirmClientAssignment` is the join; holding `Clients:ViewAll` bypasses it. Client users are pinned to their own `clientId` and can never read across.

Permission catalog (firm scope): `Users`, `Roles`, `Clients` (incl. `ViewAll`), `Services`, `Categories`, `Sales`, `Expenses`, `TaxComputation`, `TaxRules`, `ChartOfAccounts`, `FinancialStatements`, `Billing`, `BIRForms`, `EmailTemplates`, `Invitations`, `Reports`, `BIRFiling`, `InputTaxAsset`, `IntegrationClient`, `AuditLogs`.

Permission catalog (client scope): `ClientUsers`, `Categories:Read`, `Sales`, `Expenses`, `TaxComputation:Read`, `Reports`, `BIRFiling:Read`.

Roles are editable in the UI (**Roles editor**), with system roles protected from deletion.

---

3. Architecture & tech stack

A **pnpm monorepo**, three workspaces, deployed as **two Docker services on Sliplane**.

```
Accounting-Firm-Portal/
├── apps/api          NestJS backend  → Sliplane service "api"
├── apps/web          React frontend  → Sliplane service "web"
├── packages/shared   @portal/shared — frozen enums + integration contract
└── docs/             system-design.md · bir-integration-spec.md · ROADMAP.md
                      DEPLOY-SLIPLANE.md · BUILD-PLAN.md (this file)
```

Frontend — `apps/web`

React 18 · TypeScript (strict) · Vite · Tailwind (navy/gold design system) · Radix/shadcn primitives · TanStack Query + TanStack Table · React Hook Form + Zod · Recharts · SheetJS (xlsx) + PapaParse (csv) · `html2canvas` + `jsPDF` for DOM→PDF · Vitest + Playwright.

Backend — `apps/api`

Node 22 · TypeScript (strict, `noUncheckedIndexedAccess` **on**) · **NestJS** · REST/JSON at `/api/v1` · OpenAPI/Swagger at `/api/v1/docs` · **Prisma** · Jest + Supertest.

Runtime quirk that matters: the API runs on **`ts-node`**, *not* `tsx`/esbuild. esbuild strips `emitDecoratorMetadata`, which NestJS dependency injection requires. A `ts-node` `moduleTypes` override compiles the ESM-source shared package as CJS.

Shared — `packages/shared` (`@portal/shared`)

Consumed **as source** (its `main` points at `src/index.ts`); Vite transpiles it for web, a tsconfig `paths` alias resolves it for the API. Holds the **frozen tax-classification enums** and the full Portal ⇄ Generator contract schemas.

Data & infrastructure

Concern	Choice
Database	PostgreSQL (Prisma; 33 migrations, 35 models)
Cache/queue	Redis — wired up, **BullMQ not currently used**

Concern	Choice
Object storage	S3-compatible (Cloudflare R2 / MinIO) — COR files, avatars, exports
Auth (users)	argon2 password hashing + JWT + **TOTP MFA**
Auth (machines)	**OAuth2 client-credentials** with scopes
SSO	Google + Microsoft **OIDC** (sign-in only, never self-provisioning)
Email	Provider-agnostic adapter — **Postal** (self-hosted) or **Plunk**
Observability	**Sentry** (errors + OTel-compatible tracing), opt-in via `SENTRY_DSN`
CI	GitHub Actions → Sliplane

---

4. Non-negotiable guardrails

These are enforced in code and in review. Violating one is a bug, not a preference.

1. **Authoritative BIR tax lives only in the BIR Forms module.** The bookkeeping side (`tax-estimate`, `vat-summary`, transaction aggregates) is a **management estimate** and must **never override** a figure computed by a generated form. The wall between *estimate* and *authoritative* stands — it just runs inside the Portal now rather than across a service boundary.
2. **Tax-classification enums are frozen** and defined once in `@portal/shared`: `VatClass`, `InputVATCategory`, `InputTaxAttribution`. Import them; never retype them inline anywhere.
3. **Transaction amounts are stored net of VAT.** VAT is carried in its own fields.
4. **Integration write endpoints are idempotent**, keyed by `client + form + period` (upsert, never duplicate).
5. **The Portal supplies amounts only** for percentage tax — the ATC and the rate are owned by the form generator. Never send a rate.
6. **Secrets never reach the browser.** Only `VITE_`-prefixed variables are bundled. Machine integrations authenticate server-to-server.

---

5. Domain model — all 35 entities

Tenancy & identity

Model	Role
`Firm`	The tenant root. Everything hangs off a `firmId`.
`Client`	A client organisation of the firm — TIN, RDO, registration facts, tax regime.
`User`	A login. Discriminated by `userType`: `FIRM` or `CLIENT`.
`FirmUserProfile`	Firm-staff profile (name, avatar, contact).
`ClientUserProfile`	Client-portal user profile, pinned to a `clientId`.
`FirmClientAssignment`	Which firm staff may reach which client. The scope join.
`Invitation`	Tokenised invite to create an account; TTL-bounded.

Authorisation & audit

Model	Role
`Role`	Named permission bundle, scoped `FIRM` or `CLIENT`. Editable; system roles protected.
`Permission`	A `Resource:Action` pair.
`UserRole`	User → Role.
`RolePermission`	Role → Permission.
`IntegrationClient`	OAuth2 machine credential with granted scopes.
`AuditLog`	Append-only record of who did what to which entity, with metadata.

Bookkeeping

Model	Role
`Category`	Income/expense category, firm- or client-scoped.
`IncomeTransaction`	A sale. Net-of-VAT amount + separate VAT fields + `VatClass`.
`PurchaseTransaction`	A purchase. Net amount, VAT, `InputVATCategory`, `InputTaxAttribution`.
`InputTaxAsset`	Capital goods whose input VAT amortises across periods.

Chart of Accounts & tax reference

Model	Role
`ChartAccount`	Seeded catalogue with an authoritative hierarchy; CRUD + archive/restore.
`AccountTaxMapping`	Maps a chart account to a tax line.
`BirTaxType`	BIR reference: tax types.
`BirAtcCode`	BIR reference: the ATC (Alphanumeric Tax Code) table.
`TaxRule`	Configurable brackets/rates/strategy driving the tax **estimate**.

BIR forms & filings

Model	Role
`BirForm`	A saved form instance — form code, client, period, input data, status (`draft` / `filed`), `filedAt`.
`BirFormExport`	A generated artifact (eBIRForms XML) for a form.
`BIRFiling`	A filing record; publishes a filed form's authoritative figures onto the client tax view.

Services & billing

Model	Role
`Service`	The firm's service catalogue with default pricing/tax treatment.
`Invoice`	A billing. Status: `Draft` → `Sent` → `Paid` (terminal), plus `Overdue`.

Model	Role
`InvoiceLineItem`	A billed line with its own tax selection.
`BillingCounter`	Per-firm sequence for billing numbers.

Financial Statement Creator

Model	Role
`FsReport`	A statement set (BS / IS / CF / CE + Notes) for a client + period.
`FsPeriod`	The reporting period and its comparatives.
`TrialBalanceEntry`	Imported/derived trial-balance line.
`FsAdjustment`	An adjusting entry.
`FsAdjustmentLine`	A debit/credit line of an adjustment.
`FsNote`	A note to the financial statements.

---

6. Feature inventory — everything that is built

6.1 Authentication & accounts

- Email + password login (**argon2**), JWT access tokens, configurable TTLs.
- **TOTP MFA** with a short-lived MFA-challenge token.
- **Google & Microsoft OIDC SSO** — authorization-code flow. **Signs in existing accounts only**, matched by verified email. There is no self-provisioning path; a provider's button appears on the login page only when its client id/secret *and* `API_PUBLIC_URL` are configured.
- **Invitations** for both firm staff and client users, tokenised with a TTL (default 7 days), emailed, with a resend path when delivery fails.
- **User profiles** with avatar upload to object storage.
- Password reset / change and email-verification templates exist.

6.2 Client management

- Client CRUD with archive/restore (`portal_set_client_status`).
- Registration facts: TIN, branch code, RDO, registered name/address, line of business, tax regime (VAT / non-VAT / 8%), registered tax types.
- **COR upload + OCR auto-fill** — upload the BIR Certificate of Registration and have the onboarding form populated from it. Parsing runs client-side (`lib/cor/parseCor.ts`); the file itself goes to S3-compatible storage (optional — the routes return 503 when storage is unconfigured and everything else keeps working).
- **Sub-clients** — a client can sit under a parent, including for consolidated billing.
- Per-client assignment of firm staff.

6.3 Bookkeeping — sales & expenses

- Regime-aware capture forms driven by the frozen enums.

- **Amounts net of VAT**, VAT carried separately (guardrail #3).
- Categories, both firm-level and client-level.
- **CSV/XLSX import** with a client-side parse + preview (`spreadsheet.ts`, `ImportModal`) and row-level validation, posted to `POST /import` endpoints that validate and commit in-request. **Synchronous** — see §13.
- **Export** to CSV/XLSX.
- Input-tax assets with amortisation.

6.4 Tax estimation

- Configurable **tax rules** (brackets, rates, strategy method) per client.
- A client **Tax page** showing the estimate — clearly a management guide.
- When a BIR form is **filed**, its authoritative figures publish onto that same view, which is where guardrail #1 becomes visible to the user: the estimate never overwrites a generated form's number.

6.5 Chart of Accounts

- Seeded catalogue with an authoritative hierarchy registry.
- CRUD, archive/restore.
- Account → tax-line mappings.

6.6 Financial Statement Creator

- Balance Sheet, Income Statement, Cash Flow, Changes in Equity, plus **Notes**.
- Built from client data and trial-balance entries, with adjusting entries.
- **Formula-bearing `.xlsx` export** (real formulas, not flattened values).

6.7 BIR Forms

See §7 — the largest single subsystem.

6.8 Services & billing

See §8.

6.9 Client Portal

Client users get their own navigation shell and a read-scoped view:

- **Home** — their dashboard.
- **Sales** / **Expenses** — their own transactions (create/edit/export per role).
- **Tax** — their tax position.
- **Filings** — their BIR filings.
- **Users** — Client Owners manage their own portal users.

6.10 Audit log

Every meaningful mutation is recorded with actor, action, entity, and metadata — including writes that arrive through the MCP endpoint (logged as **"Claude (MCP)"**) and through the OAuth2 integration endpoints.

6.11 Documents & settings

A **Settings hub** with tabs: Users & Roles, Documents, Integrations, Audit Log, and Email & Senders. Legacy top-level paths (`/users`, `/audit`, `/documents`, `/integrations`) redirect into it.

---

7. The BIR Forms module (internal Generator)

This is the biggest thing in the codebase and the biggest departure from the original plan. The original design put BIR form layout, eBIRForms XML, and authoritative tax math in a *separate* system reached over OAuth2. That system (the "Sentire BIR Form Generator") was instead **ported into the Portal** at `apps/api/src/bir-forms` plus its Firm Admin UI.

The OAuth2 contract in `docs/bir-integration-spec.md` is **retained as the module's seam**, so a standalone Generator product ("Sentire Tax") could still integrate later. Nothing in the Portal depends on that happening.

All nine forms are live

Form	What it is	Output
2551Q	Quarterly Percentage Tax	eBIRForms XML
2550Q	Quarterly VAT	eBIRForms XML
1701Q	Quarterly Income Tax — individuals	eBIRForms XML
1701A	Annual Income Tax — 8% / OSD, single income source	eBIRForms XML
1701	Annual Income Tax — mixed income	eBIRForms XML
1702Q	Quarterly Income Tax — corporations	eBIRForms XML
1702RT	Annual Income Tax — corporations, regular rate	eBIRForms XML
2307	Certificate of Creditable Tax Withheld at Source	**Print to PDF**
2316	Certificate of Compensation Payment / Tax Withheld	**Print to PDF**

Why 2307 and 2316 print instead of exporting XML: they are *certificates issued to a payee*, not returns you e-file. BIR defines no eBIRForms XML for them. The service rejects an export attempt explicitly rather than emitting a bogus file.

Engine structure

`apps/api/src/bir-forms/engine/` — deliberately flattened. Sentire's `lib/compute/*`, `lib/xml/*`, `lib/format`, `lib/taxTables`, `lib/period`, and `lib/xml/xmlkit` all sit side-by-side:

- `types.ts`, `format.ts`, `period.ts`, `taxTables.ts`, `xmlkit.ts`, `index.ts` (barrel)
- Per form: `compute<FORM>.ts` + `build<FORM>.ts`, each with a `.spec.ts`

`bir-forms.service.ts` is the dispatcher, holding `AVAILABLE_FORMS` (all nine) and `XML_EXPORT_FORMS` (the seven returns), and exposing compute / buildXml / keyFigures / export.

The XML format

eBIRForms packages are rows of `<div>KEY=VALUEKEY=</div>` with a per-form namespace and a form-specific assembly and tail, plus a canonical filename built from TIN + branch + form

- period.

Parity testing

Every ported form carries a parity test against a real eBIRForms export — namespace, field keys, package quirks, tail, and canonical filename. The 1701 spec asserts the exact **837-row** count and the filename `2184305230001701v2018122025.xml`. This is the highest-value test surface in the repo and it is where regressions get caught.

Filing lifecycle

A form is `draft` until marked **filed** (`filedAt` stamped). Filing publishes its figures onto the client's tax view. Drafts and filed forms are listed and filterable on the BIR Forms page.

⚠️ The known gap: the Form view & live PDF preview

Sentire's Generator had **two layers per form**:

1. **Guided** — a data-entry wizard.
2. **Form** — a pixel-faithful replica of the printed BIR form, driving a **live PDF preview**.

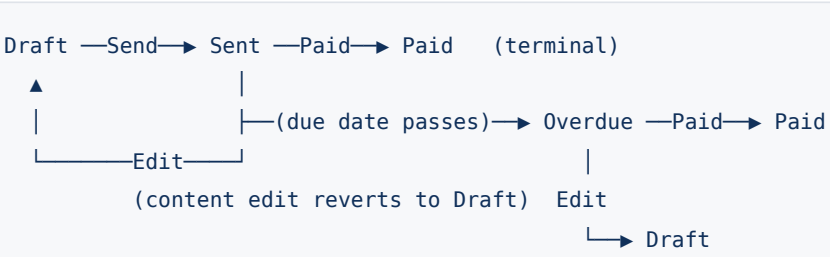
Only Guided was ported. The Form layer was deferred during the port and not returned to — this was a real gap and the user was right to flag it. Groundwork is merged (`apps/web/src/components/birform/` — `formkit.tsx`, `formparts.tsx`, `formProps.ts`, `types.ts`, `format.ts`, plus `styles/bir-form.css` and `lib/sheetPdf.ts`), but **nothing imports it yet.** See §14.

---

8. Billing lifecycle

Billing is **centralised at the firm level** (`/billing`), not per-client. Old per-client billing URLs redirect.

States



Rule	Detail
Sendable	`Draft`, `Overdue`
Editable	`Draft`, `Sent`, `Overdue`
Edit reverts to Draft	Editing the *content* of a `Sent` or `Overdue` billing pulls it back to `Draft`. The audit metadata records `revertedToDraftFrom`.
Payable	`Sent`, `Overdue`
Paid is permanent	There is no "Unpaid" reversal. `Paid` is terminal and rejects any further change: *"This billing is marked Paid and can no longer be changed. Raise a new billing instead."*

"Content" means `description`, `issuedDate`, `dueDate`, or `lineItems` — a status-only update does not trigger the revert.

The terminal rule is enforced server-side, in `InvoicesService.update()`, not only in the UI — so it also holds for the MCP tools and for any direct API call.

Other billing features

- Per-line **tax selection**; services carry defaults that pre-fill lines.
- Auto-numbered via `BillingCounter`.
- **Row click opens a read-only preview** of the rendered billing document (A4, natural 794px width; Escape closes).
- **PDF / JPEG export** of the billing document.
- **Sub-client billing** rolled up under a parent client.

Email on status change

Transition	Email sent?
Send (`Draft` / `Overdue` → `Sent`)	✅ Yes — the billing statement goes to the client.
Mark Paid	❌ No.
Edit → revert to `Draft`	❌ No — deliberately, so correcting a billing does not spam the client.

A `paymentReceivedEmail` template **exists but is not wired**. Wiring it to the Paid transition is a small, available change if the firm wants a receipt/confirmation.

---

9. Email

Provider-agnostic by adapter, selected with `MAIL_PROVIDER`:

- `postal` (default) — the firm's self-hosted Postal server.
- `plunk` — alternative adapter.

Templates are hand-built in `apps/api/src/mail/email-templates.ts` against a shared theme in `email-theme.ts`. Delivery is logged. When mail is unconfigured, invitations are still created and show *"email failed"* with a **Resend** button rather than failing the flow.

Templates

Wired and sending today (4): `staffInviteEmail` · `clientInviteEmail` · `roleChangedEmail` · `invoiceDueEmail` (the billing statement, sent on Send).

Written but not yet wired (13): `welcomeClientEmail` · `welcomeStaffEmail` · `emailVerificationEmail` · `passwordResetEmail` · `passwordChangedEmail` · `documentRequestEmail` · `documentReadyEmail` · `esignRequestEmail` · `paymentReceivedEmail` · `deadlineReminderEmail` · `appointmentConfirmationEmail` · `messageNotificationEmail` · `returnFiledEmail`

These are ready-made hooks — each represents a feature that is one wiring change away (payment receipts, filing deadline reminders, document requests, e-sign requests, appointment confirmations).

---

10. Integrations

10.1 OAuth2 client-credentials (machine-to-machine)

`IntegrationClient` records hold a key/secret and a set of granted **scopes**. Integration endpoints check the scope in addition to normal auth, and all writes are audit-logged. Write receivers (`BIRFiling` , `InputTaxAsset`) are **idempotent, keyed by client + form + period** (guardrail #4).

This is the retained seam for a future standalone Generator ("Sentire Tax"). The full contract — entities, endpoints, JSON shapes, enums, aggregation rules — is in `docs/bir-integration-spec.md` .

10.2 MCP endpoint (Claude machine access)

`POST <api-base>/api/v1/mcp/<secret>` exposes firm-scoped tools so Claude (claude.ai or a Cowork custom connector) can operate the Portal directly.

Read tools: `portal_list_clients` , `portal_get_client` , `portal_list_invoices` , `portal_list_income_transactions` , `portal_list_expense_transactions` , `portal_list_transaction_categories` , `portal_financial_summary` .

Write tools: `portal_create_client` , `portal_update_client` , `portal_set_client_status` , `portal_record_income` , `portal_record_expense` , `portal_delete_transaction` , `portal_create_invoice` , `portal_update_invoice` , `portal_update_invoice_status` .

Writes run through **the same service layer as the web UI** — so the billing terminal-Paid rule, RBAC, and validation all apply identically — and are audit-logged as **"Claude (MCP)"**.

Security: the secret is portal-managed (viewed/rotated/disabled by a Super Admin on the Integrations page). Anyone holding the URL can **write** portal data; treat it as a password. `MCP_SHARED_SECRET` in env is only a pre-portal fallback and is ignored once rotated from the UI. Leave it unset and the route returns 404 (feature off). Minimum 32 characters.

10.3 SSO

Google and Microsoft OIDC — see §6.1.

10.4 Object storage

S3-compatible (R2/MinIO) for COR files, avatars, and exports. Fully optional: unset and the dependent routes return 503 while everything else runs.

---

11. Application surface — routes & API modules

Web routes (40 pages)

Public: `/login` · `/accept` · `/sso/callback`

Firm (inside `AppShell`):

Route	Page
`/`	Dashboard (client users are redirected to `/portal`)
`/profile`	Profile

Route	Page
`/settings` → `/settings/users`	Settings hub index
`/settings/users` · `/settings/documents` · `/settings/integrations` · `/settings/audit` · `/settings/email`	Settings tabs
`/clients` · `/clients/new` · `/clients/:id` · `/clients/:id/edit`	Client roster & record
`/clients/:id/sales` · `/expenses` · `/tax` · `/tax-rules` · `/filings`	Per-client workspace
`/services`	Service catalogue
`/billing`	Centralised billing
`/chart-of-accounts`	Chart of Accounts
`/financial-statements` · `/financial-statements/:id`	FS Creator
`/bir-forms` · `/bir-forms/new` · `/bir-forms/:id`	BIR Forms list & editor

Client portal: `/portal` · `/portal/sales` · `/portal/expenses` · `/portal/tax` · `/portal/filings` · `/portal/users`

Per-form editors exist for all nine forms (`BirForm2551QEditor` … `BirForm2316Editor`), dispatched from `BirFormEditorPage`.

API modules (33)

`audit` · `auth` · `bir` · `bir-forms` · `categories` · `clients` · `coa` · `common` · `dashboard` · `files` · `filings` · `financial` · `fs` · `health` · `income-transactions` · `integration` · `invitations` · `invoices` · `mail` · `mcp` · `observability` · `portal` · `prisma` · `profile` · `purchase-transactions` · `rbac` · `redis` · `roles` · `services` · `settings` · `storage` · `tax-rules` · `users`

Health at `/api/v1/health`; Swagger at `/api/v1/docs`.

Test coverage

453 API unit tests · 99 web unit tests · 6 API e2e (Supertest) · Playwright E2E covering login, home, financial capture, and BIR Forms. All hermetic — no database needed for the unit or e2e suites.

---

12. Security posture

Control	State
Password hashing	argon2
MFA	TOTP, enforced through a short-lived challenge token
Session	JWT, configurable TTL
Authorisation	Data-driven RBAC + per-client assignment scope, checked on **every** endpoint
Machine auth	OAuth2 client-credentials with scope checks
Audit	Append-only log covering UI, MCP, and integration writes

Control	State
Secrets in browser	Only `VITE_`-prefixed vars are bundled (guardrail #6)
Error reporting PII	`sendDefaultPii: false` — stack + route only, never request bodies or the signed-in user; only ≥500 responses are reported
Rate limiting	⬜ **Not built** — the main outstanding security item

Outstanding: rotate exposed secrets

Several credentials were pasted into chat or screenshots during development and should be rotated in Sliplane: `POSTAL_API_KEY`, `MS_CLIENT_SECRET`, the S3 access key and secret, `MCP_SHARED_SECRET`, and the Maya sandbox key. Rotating `MCP_SHARED_SECRET` is done from the Integrations page.

---

13. Build history — original plan vs. what landed

The original plan was ten phases (0–9). All ten are complete, two landed differently than designed, and a very large amount was built beyond the plan.

Legend: ✅ done · ⚠️ done differently · ⬜ not built

Phase	Planned	Status
0	Scaffold, `@portal/shared`, Prisma + Postgres, Redis, lint/test/CI, health check	✅
1	Auth (argon2 + JWT + TOTP MFA), data-driven RBAC, users, invitations, audit log	✅
2	Categories, income/purchase transactions with the frozen enums, regime-aware capture	✅
3	CSV/XLSX import/export with row-level validation	⚠️ built **synchronously**
4	Email + billing: invoices, invitations, delivery logging	⚠️ built **without MJML**
5	Tax rules, brackets, strategy methods, the client tax page (an *estimate*)	✅
6	OAuth2 client-credentials, integration clients, scoped aggregation endpoints	✅
7	`BIRFiling` + `InputTaxAsset` write receivers, idempotent upsert	✅
8	Client Portal: role-scoped dashboards and read-only visibility	✅
9	Audit coverage, rate limiting, observability, E2E, deployment	⚠️ partial

The two divergences

Phase 3 — imports are synchronous. The plan called for `ImportBatch` + `ImportError` entities processed asynchronously through **BullMQ**. What shipped is a client-side parse + preview posting to `POST /import` endpoints that validate and commit in-request. Redis is wired but **BullMQ is unused**. This is fine at the firm's current file sizes; revisit only if an import starts timing out a request.

Phase 4 — email is adapter-based, not MJML. The plan named MJML + Handlebars on SES/SendGrid. What shipped is a `MAIL_PROVIDER`-selected adapter (Postal / Plunk) with hand-built templates against a shared theme. Delivery logging is in place either way.

Phase 9 detail

Item	Status
Audit coverage on integration endpoints	✅
CI/CD (GitHub Actions → Sliplane)	✅
E2E tests	✅ login, home, financial capture, BIR Forms
Observability	✅ opt-in Sentry (errors + OTel-compatible tracing); inert until `SENTRY_DSN` is set
Rate limiting	⬜ **not built**

Built beyond the original plan

None of this appears in the original roadmap:

- **The entire BIR Form Generator** — nine forms, engine, XML builders, parity tests, and the filing lifecycle. The plan had assigned all of it to a separate system.
- **Chart of Accounts** — seeded catalogue, hierarchy registry, CRUD, archive/restore, account→tax-line mappings.
- **Financial Statement Creator** — BS/IS/CF/CE + Notes with a formula-bearing xlsx export.
- **BIR reference data** — tax types and the ATC code table.
- **Services catalogue & billing** — per-line tax selection, default-service wiring, PDF/JPEG export, sub-client billing, the full Draft→Sent→Paid lifecycle.
- **Google / Microsoft OIDC SSO.**
- **Roles editor** with system-role protection.
- **COR upload + OCR auto-fill.**
- **User profiles + avatar upload.**
- **MCP module** — 16 firm-scoped tools for machine access.

A note on process

CI was **red on `main` for months** because of a `no-useless-escape` lint error that was twice dismissed as pre-existing. `pnpm -r lint` gates CI, so every merge in that window was red and a genuine regression would have been indistinguishable from the standing failure. It was fixed (two regex escapes, verified equivalent before changing) and `main` is green. The lesson is recorded here on purpose: **a standing red build is not a cosmetic problem — it disables the signal.**

---

14. What is left to build

Ordered roughly by value.

1. BIR form view + live PDF preview — the largest open item

Sentire's pixel-faithful printed-form replicas and their live PDF preview were never ported. Groundwork is merged but unused.

Remaining work:

- The **editor shell** — a Guided ⇄ Form mode switch, zoom / fit-to-width, and a debounced live PDF preview.
- **Nine `Form*.tsx` faithful replicas** (~280 KB of layout code).
- Completing the partial `Comp*` types in `apps/web/src/lib/api.ts` into **full mirrors of the engine types** — the replicas read fields that are not currently declared.
- Replacing the hand-written 2307/2316 sheets with Sentire's real replicas.

Planned sequencing: PR 1 = shell + `Form1701`; PRs 2–3 = the remaining eight.

2. Rate limiting

The one unbuilt Phase 9 item. `@nestjs/throttler`, with tighter limits on the auth endpoints (login, MFA, SSO callback, invite accept) and on the OAuth2 token endpoint.

3. Turn on observability

Set `SENTRY_DSN` (optionally `SENTRY_ENVIRONMENT`, `SENTRY_TRACES_SAMPLE_RATE`, `SENTRY_RELEASE`) in Sliplane. The code is already deployed and inert — no network calls happen until the DSN is set.

4. Rotate exposed secrets

See §12. Operational, not a code change.

5. Wire the remaining email templates

Thirteen templates are written and unused. The highest-value ones:

- `paymentReceivedEmail` on the Paid transition (a receipt/confirmation).
- `deadlineReminderEmail` for filing deadlines.
- `documentRequestEmail` / `documentReadyEmail` for the document workflow.

6. Async imports

Only if file sizes start timing out a request. Would mean the originally planned `ImportBatch` / `ImportError` entities plus BullMQ consumers on the already-wired Redis.

7. Maya payment gateway

Investigated and sandbox-tested; **no code shipped**. Pick this up if client-facing online payment is wanted. The sandbox key that was exposed should be rotated first.

8. Sentire Tax (a standalone Generator product)

A separate, later product. If it is ever built, it integrates over the retained OAuth2 contract in `docs/bir-integration-spec.md`. **Nothing in the Portal depends on it**, and the Portal's own BIR Forms module remains the source of truth for the firm's own filings.

---

15. Operations — deployment, CI, configuration

Deployment

Two Docker services on **Sliplane**:

Service	Contents
`web`	The React build (`apps/web`) → https://acctgfirm.mcrctas.com
`api`	The NestJS server (`apps/api`)

Details in `docs/DEPLOY-SLIPLANE.md`. **After merging a change, redeploy the affected service(s)** — API-only changes need only `api`, web-only changes only `web`.

Build note: always do a clean web build. A stale `apps/web/dist` + `node_modules/.vite` once produced an identical JS hash across substantive changes, meaning a redeploy shipped old code. `rm -rf apps/web/dist node_modules/.vite && pnpm build`.

CI — `.github/workflows/ci.yml`

Two jobs on push to `main` and on every PR.

`verify` **(hermetic, no services):** install → `pnpm --filter api prisma:generate` → `pnpm -r typecheck` → `pnpm -r lint` → `pnpm -r test` → `pnpm --filter api test:e2e` → `pnpm build`

`database` **(Postgres 16 service container):** install → prisma generate → `prisma migrate deploy` → `prisma migrate status` (schema-in-sync check) → `db:seed` (idempotent)

Commands

Prereqs: Node 22 (`.nvmrc`), pnpm 10, Postgres + Redis (`docker compose up -d`). Copy `.env.example` → `.env`.

Root:

Command	What it does
`pnpm install`	Install all workspace deps
`pnpm dev`	API + web dev servers in parallel
`pnpm build`	Build `@portal/shared`, then API and web
`pnpm typecheck`	`tsc --noEmit` across all packages
`pnpm lint`	ESLint across api + web
`pnpm test`	Unit tests across all packages
`pnpm format` / `format:check`	Prettier

API — `pnpm --filter api <script>`**:** `dev` (ts-node-dev on :3000) · `start` · `build` · `test` · `test:e2e` · `prisma:generate` · `prisma:migrate` · `prisma:deploy` · `db:seed`

Web — `pnpm --filter web <script>`**:** `dev` · `build` · `preview` · `typecheck` · `lint` · `test` (Vitest) · `test:e2e` (Playwright)

> After a fresh install, run `pnpm --filter api prisma:generate` — Prisma generates into `node_modules`.

Configuration surface

Group	Variables
Core	`NODE_ENV`, `API_PORT`, `DATABASE_URL`, `REDIS_URL`
Auth	`JWT_SECRET`, `JWT_ACCESS_TTL`, `JWT_MFA_TTL`, `OAUTH_TOKEN_TTL`, `INVITE_TTL_HOURS`
Seed	`SEED_FIRM_NAME`, `SEED_ADMIN_EMAIL`, `SEED_ADMIN_PASSWORD`, optional `SEED_INTEGRATION_CLIENT_KEY` / `_SECRET`
SSO *(optional)*	`API_PUBLIC_URL`, `GOOGLE_CLIENT_ID` / `_SECRET`, `MS_CLIENT_ID` / `_SECRET`, `MS_TENANT`
Email *(optional)*	`MAIL_PROVIDER`, `MAIL_FROM`, `MAIL_FROM_NAME`, `POSTAL_API_KEY`, `POSTAL_API_URL`, `PLUNK_SECRET_KEY`, `WEB_APP_URL`
MCP *(optional)*	`MCP_SHARED_SECRET` (fallback only; portal-managed once rotated)
Storage *(optional)*	`S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `S3_REGION`
Web	`VITE_API_BASE_URL` — **the only browser-visible group**
Observability *(optional)*	`SENTRY_DSN`, `SENTRY_ENVIRONMENT`, `SENTRY_TRACES_SAMPLE_RATE`, `SENTRY_RELEASE`

Every optional group **degrades gracefully**: unset SSO hides the buttons, unset mail keeps invitations working with a resend button, unset storage returns 503 on COR routes only, unset MCP returns 404, unset Sentry never initialises.

---

16. Working conventions for contributors

Code

- **TypeScript strict everywhere.** No `any` without a comment justifying it.
- `noUncheckedIndexedAccess` is **on** in the API — indexed reads need `?? fallback` guards. Ported code frequently needs this.
- **Validation lives in shared Zod schemas**; API DTOs derive from them. Enums are defined once in `@portal/shared` and imported — never retyped inline.
- REST resources under `/api/v1`; **document every endpoint in OpenAPI**.
- **Every endpoint enforces auth + per-client RBAC.** Integration endpoints also check OAuth scopes.
- Write tests for anything with real logic. The **aggregation** that turns classified transactions into `vat-summary` / `percentage-tax-summary`, and the **BIR form parity tests**, are the highest-value coverage in the repo.

The verification ritual — run before every ship

```
pnpm -r typecheck
pnpm -r lint
pnpm -r test
pnpm --filter api test:e2e
# seedcheck — full tsc over seed + src
#   apps/api/tsconfig.seedcheck.json:
#   {"extends":"./tsconfig.json","include":["prisma/seed.ts","src/**/*.ts"]}
npx tsc -p tsconfig.seedcheck.json --noEmit && rm -f tsconfig.seedcheck.json
pnpm build
```

> **Why the seedcheck matters:** Jest uses ts-jest in *transpile* mode and is lenient about some type errors. Full `tsc` (seedcheck / build) catches them. Several real errors — a bad cast in `buildXml`, a `noUncheckedIndexedAccess` destructure — passed Jest and failed `tsc`. Do not skip it.

Git workflow

Development happens on `claude/accounting-portal-kickoff-eo7zt1`. Each change gets its own PR against `main` and is squash-merged; the branch is then reset to `origin/main`. After a merge, redeploy the affected Sliplane service(s).

Documentation map

File	Contents
`CLAUDE.md`	Project memory — stack, conventions, guardrails, commands. Kept short and stable.
`docs/system-design.md`	Full system design: requirements, actors, RBAC, domain model, activity flows.
`docs/bir-integration-spec.md`	The retained Portal ⇄ Generator API contract.
`docs/ROADMAP.md`	Phased plan and what landed against it.
`docs/DEPLOY-SLIPLANE.md`	Deployment.
`docs/BUILD-PLAN.md`	**This file** — the complete picture.

Appendix — one-page summary

What: A multi-tenant portal an accounting firm runs for its own practice: clients, bookkeeping, tax estimation, authoritative BIR form generation, and a client self-service portal.

Stack: pnpm monorepo · NestJS + Prisma + PostgreSQL (`apps/api`) · React 18 + Vite (`apps/web`) · `@portal/shared` for frozen enums · Redis · S3-compatible storage · two Docker services on Sliplane.

Scale: 35 Prisma models · 33 migrations · 33 API modules · 40 web pages · 453 API + 99 web unit tests · 6 API e2e · Playwright E2E · CI green.

The big idea: the bookkeeping side produces a *management estimate*; the BIR Forms module produces the *authoritative filed figures*, and the estimate never overrides it.

Biggest thing built: the BIR Form Generator — all nine forms (2551Q, 2550Q, 1701Q, 1701A, 1701, 1702Q, 1702RT as eBIRForms XML; 2307, 2316 as print-to-PDF certificates), each with a parity test against a real eBIRForms export.

Biggest thing left: the pixel-faithful Form view and its live PDF preview — Sentire's second layer per form, which was never ported.

Also left: rate limiting · turn on Sentry · rotate exposed secrets · wire 13 written-but- unused email templates · async imports (only if needed) · Maya gateway · Sentire Tax.